LABORATORIO 7: Introducción a la visión por computadora

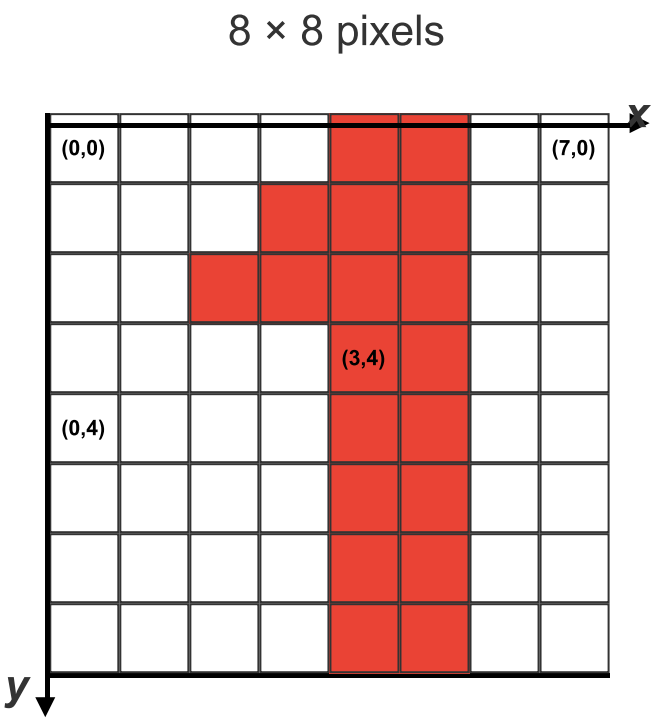
Conceptos Teóricos:

Parte 1: Visión por Computadora.

La visión por computadora permite a diferentes sistemas procesar imágenes y video con distintos propósitos. Existen muchas aplicaciones para sistemas embebidos que utilizan la visión por computadora para generar una respuesta o interactuar con los cambios del entorno.

- En tu informe de laboratorio, menciona 2 proyectos que utilicen visión por computadora embebida.

Una de las bibliotecas más utilizadas para trabajar con sistemas embebidos es OpenCV. Esta biblioteca se puede encontrar en diferentes lenguajes de programación y facilita las distintas transformaciones de píxeles que un desarrollador necesita implementar para un proyecto.



Como podemos ver en la imagen, cada imagen tiene una cantidad diferente de píxeles; esto determina el tamaño de la imagen y funciona como una matriz que puede ser afectada por diferentes operaciones.

¿Qué es un píxel? ¿Por qué lo usamos para medir las resoluciones de imagen?

¿Qué valores se pueden asignar a un píxel? ¿Qué determina el valor de un píxel?

Para trabajar con OpenCV usaremos la importación de módulos en python. Esta es una técnica común utilizada para modularizar proyectos grandes y evitar un alto acoplamiento con baja cohesión.

Primero que nada necesitamos instalar opencv, para ello podemos revisar el comando requerido en la página web oficial de OpenCV:
https://opencv.org/get-started/

Ahora para usar OpenCV haremos lo siguiente:

- Importar la biblioteca
- Usar la función `imread()`
- Realizar procesamiento de imágenes
- Usar la función `imshow()`

Revisemos el siguiente ejemplo:

```
      import cv2 def resize_img(img, width, height): up_points = (width, height)
img_resize = cv2.resize(img, up_points) return img_resize if __name__ ==
"__main__": img = cv2.imread("assets\img1.jpg") rimg = resize_img(img, 500, 500)
cv2.imshow("resize image", rimg) cv2.waitKey(0)
```

¿Cómo funciona este código?

¿Qué hace `__name__ == "__main__"`?

Para trabajar adecuadamente con OpenCV es importante utilizar buenas prácticas relacionadas con la programación en python e implementarlas para mejorar nuestro código.

Ejercicios:

1. Crea un programa que abra una imagen y cambie su tamaño a 1000x1000 píxeles.
2. Genera un programa que pida al usuario seleccionar entre tamaño original, pequeño, mediano o grande, y reescale una imagen.

3. Crea un programa que rote 90 grados una imagen cada vez que el usuario presione cualquier tecla del teclado. ¿Cuál es la diferencia entre imprimir una imagen y usar `imshow`?
4. Usando OpenCV lee una imagen y realiza las siguientes operaciones:
   - Lee la imagen y muéstrala.
   - Redimensiona la imagen a 400×600 (ancho, alto).
   - Corta la imagen horizontalmente y muestra las dos mitades.
   - Corta la imagen verticalmente y muestra las dos mitades.
   - Divide la imagen en cuadrantes del mismo tamaño y muéstralos con el título "Cuadrante N" de acuerdo al número de cuadrantes.

Con OpenCV también podemos abrir nuestra cámara web y los videos almacenados en nuestro sistema, para ello usaremos la función `VideoCapture()`. Para esta función podemos indicar un número (0 para la cámara predeterminada) o la ubicación de un archivo de video, veamos el siguiente ejemplo:

```
      import cv2 from abc import ABC, abstractmethod class video_Capture_abs(ABC):
@abstractmethod def display_camera(self): pass @abstractmethod def
stop_display(self): pass @abstractmethod def camera_visualization(self): pass
```

```
      class video_Capture(video_Capture_abs): def __init__(self, camera) -> None:
self.camera = camera self.displayed = False def display_camera(self):
self.displayed = True self.camera_visualization() def stop_display(self):
self.displayed = False def camera_visualization(self): while self.displayed:
check, frame = self.camera.read() cv2.imshow("camera", frame) key = cv2.waitKey(1)
if key == 27: # 27 es el código ASCII para la tecla 'Esc' self.stop_display() if
__name__ == "__main__": camera = cv2.VideoCapture(0) camera_object =
video_Capture(camera) camera_object.display_camera()
```

Como podemos ver, `VideoCapture` nos permite seleccionar la cámara. Sin embargo, para obtener la información real de los fotogramas (frames) necesitamos usar la función `.read()` en el objeto de la cámara.

- ¿Qué pilares de la Programación Orientada a Objetos (OOP) se utilizaron en el código anterior? ¿Por qué?
- ¿Qué valor obtiene la variable `check` del método `camera_visualization`?
- Investiga cómo cambiar el filtro de color para una imagen y un video usando OpenCV.

Ejercicios (Continuación):

1. Crea una carpeta llamada "colors" con 3 imágenes de 3 colores diferentes. Luego, usando OpenCV abre cada imagen e imprime el valor del color en la terminal.
2. A partir del ejercicio anterior, haz una función que convierta el color rgb a escala de grises.
3. Escala el ejercicio anterior con una clase que pueda modificar el color original de una imagen a RGB, HSV y escala de grises. Luego muestra la imagen modificada. ¿Qué es HSV?

4. Usando HSV, detecta el color rojo y muéstralo en la consola, luego el verde y el azul. ¿Cómo puedes pasar estos valores a TIVA?
5. Abre una cámara web usando OpenCV y muestra el fotograma en escala de grises y luego en RGB. Cambia entre escala de grises y RGB usando las teclas del teclado.
6. Investiga cómo capturar la imagen de un fotograma y guardarla como una imagen usando OpenCV. Luego, captura la imagen de la cámara web y guarda el fotograma cada vez que se presione la tecla "c" con el formato "Captures/image1.jpg". Cuando el proceso termine, aplica un filtro de escala de grises a la imagen y divídela en cuadrantes.